

Week 3 - Module

→ Topic 1 : - Merge Sort

* Question 1 (Easy) - Rank students by marks

⇒ #include <stdio.h>

#include <stdlib.h>

```
void merge (int arr[], int left, int mid, int right) {  
    int n1 = mid - left + 1;  
    int n2 = right - mid;
```

```
    int *L = (int*) malloc (n1 * sizeof (int));
```

```
    int *R = (int*) malloc (n2 * sizeof (int));
```

```
    for (int j = 0; j < n1; j++) L[j] = arr[left + j];
```

```
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
```

```
    int i = 0, j = 0, k = left;
```

```
    while (i < n1 & j < n2) {
```

```
        if (L[i] <= R[j]) arr[k++] = L[i++];
```

```
        else arr[k++] = R[j++];
```

```
    }
```



```
while (i < n1) arr[k++] = L[i++];  
while (j < n2) arr[k++] = R[j++];
```

```
free(L);  
free(R);  
}
```

```
void mergesort(int arr[], int left, int right)  
{  
    if (left < right) {  
        int mid = left + (right - left) / 2;  
        mergesort(arr, left, mid);  
        mergesort(arr, mid + 1, right);  
        merge(arr, left, mid, right);  
    }  
}
```

```
int *main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int *marks = (int *) malloc (n * size of (int));
```

```
    for (int i = 0; i < n; i++) scanf("%d", &marks[i]);
```

```
    mergesort(marks, 0, n - 1);
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("%d", marks[i]);
```



```
if (i != n-1) printf(" ");
```

```
}
```

```
printf("\n");
```

```
free(marks);
```

```
return 0;
```

```
}
```

* Question 2 (Easy)

→ organizing Product Prices

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void merge (int arr[], int left, int mid, int right){
```

```
int n1 = mid - left + 1;
```

```
int n2 = right - mid;
```

```
int* L = (int*) malloc (n1 * sizeof (int));
```

```
int* R = (int*) malloc (n2 * sizeof (int));
```

```
for (int i = 0; i < n1; i++) L[i] = arr[left + i];
```

```
for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
```



```

int i = 0, j = 0, k = left;
while (i < n1 || j < n2) {
    if (L[i] <= R[j]) arr[k++] = L[i++];
    else arr[k++] = R[j++];
}

```

```

while (i < n1) arr[k++] = L[i++];
while (j < n2) arr[k++] = R[j++];

```

```

free(L);
free(R);

```

```

}

```

```

void mergesort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergesort(arr, left, mid);
        mergesort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

```

```

int main() {
    int n;

```

```

    scanf("%d", &n);

```

```

    int* prices = (int*) malloc(n * size of (int));

```



```
for (int i = 0; i < n; i++) scanf("%d", &prices[i]);
```

```
mergeSort (prices, 0, n-1);
```

```
for (int i = 0; i < n; i++) {
```

```
    printf("%d", prices[i]);
```

```
    if (i == n-1) printf(" ");
```

```
}
```

```
printf("\n");
```

```
free (prices);
```

```
return 0;
```

```
}
```

* Question 3 (Medium)

→ Sort Employee records by salary

3 #include <stdio.h>

#include <stdlib.h>

```
void merge (long long arr[], int left, int mid, int right) {
```

```
    int n1 = mid - left + 1;
```

```
    int n2 = right - mid;
```



```
long long *L = (long long *) malloc (n1 * sizeof (long long));
long long *R = (long long *) malloc (n2 * sizeof (long long));
```

```
for (int i = 0; i < n1; i++) L[i] = arr[left + i];
for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
```

```
int i = 0, j = 0, k = left;
while (i < n1 && j < n2) {
    if (L[i] <= R[j]) arr[k++] = L[i++];
    else arr[k++] = R[j++];
}
```

```
free (L);
```

```
free (R);
```

```
}
```

```
void mergesort (long long arr[], int left, int right) {
    if (left < right) {
```

```
        int mid = left + (right - left) / 2;
```

```
        mergesort (arr, left, mid);
```

```
        mergesort (arr, mid + 1, right);
```

```
        merge (arr, left, mid, right);
    }
}
```



```
int main () {
```

```
    int n ;
```

```
    scanf ("%d", &n);
```

```
    long long *salaries = (long long*) malloc (n * sizeof (long));
```

```
    for (int i = 0; i < n; i++) scanf ("%lld", &salaries[i]);
```

```
    merge sort (salaries, 0, n-1);
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf ("%lld", salaries[i]);
```

```
        if (i != n-1) printf (" ");
```

```
    }
```

```
    printf ("\n");
```

```
    long long lowest = salaries[0];
```

```
    long long highest = salaries[n-1];
```

```
    long long difference = highest - lowest;
```

```
    printf ("lowest salary : %lld\n", lowest);
```

```
    printf ("highest salary : %lld\n", highest);
```

```
    printf ("difference : %lld\n", difference);
```

```
    free (salaries);
```

```
    return 0;
```

```
}
```


• Topic 2 : Quick sort

→ Question 4 (Easy)

⇒ Race Timing Analysis

* #include <stdio.h>

#include <stdlib.h>

void swap (int *a, int *b) {

int temp = *a;

*a = *b;

*b = temp;

}

int partition (int arr[], int low, int high) {

int pivot = arr[high];

int i = low - 1;

for (int j = low ; j < high ; j++) {

if (arr[j] <= pivot) {

i++;

swap (&arr[i], &arr[j]);

}

}


```
swap (& arr[i+1], & arr[high]);  
return i+1;
```

```
}
```

```
void quicksort (int arr[], int low, int high) {  
    if (low < high) {
```

```
        int pi = partition (arr, low, high);
```

```
        quicksort (arr, low, pi-1);
```

```
        quicksort (arr, pi+1, high);  
    }
```

```
}
```

```
}
```

```
int main () {
```

```
    int n;
```

```
    scanf ("%d", &n);
```

```
    int * timings = (int *) malloc (n * size of (int));
```

```
    for (int i=0; i<n; i++) scanf ("%d", &timings[i]);
```

```
    quicksort (timings, 0, n-1);
```

```
    for (int i=0; i<n; i++) {
```

```
        printf ("%d", timings[i]);
```

```
        if (i != n-1) printf (" ");
```

```
    }
```

```
    printf ("\n");
```


classmate

Date _____

Page _____

free timings) ;

return 0;

→ Question 2 (Easy)

* organizing library Book IDs

→ #include <stdio.h>

#include <stdlib.h>

```
void swap (long long *a, long long *b) {
```

```
    long long temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
int partition (long long arr[], int low, int high)
```

```
    long long pivot = arr[high];
```

```
    int i = low - 1;
```

```
    for (int j = low; j < high; j++) {
```


Page _____

```
if (arr[j] <= pivot) {  
    i++;
```

```
    swap (&arr[i], &arr[j]);
```

```
}
```

```
}
```

```
swap (&arr[i+1], &arr[high]);
```

```
return i+1;
```

```
}
```

```
void quicksort (long long arr[], int low, int high)
```

```
{  
    if (low < high) {
```

```
        int pi = partition (arr, low, high);
```

```
        quicksort (arr, low, pi-1);
```

```
        quicksort (arr, pi+1, high);
```

```
    }
```

```
}
```

```
int main () {
```

```
    int n;
```

```
    scanf ("%d", &n);
```

```
    long long *ids = (long long *) malloc (n * sizeof (long long));  
    for (int i = 0; i < n; i++) scanf ("%lld", &ids[i]);
```

```
    quicksorts (ids, 0, n-1);
```

```
    for (int i = 0; i < n; i++) {
```



```

printf("%d", ids[i]);
if (i == n - 1) printf(" ");
}

```

```

printf("\n");

```

```

free(ids);

```

```

return 0;

```

```

}

```

→ Question 3 (Medium)

* online store sales Ranking

⇒ #include <stdio.h>
#include <stdlib.h>

```

void swap (long long *a, long long *b) {

```

```

    long long temp = *a;

```

```

    *a = *b;

```

```

    *b = temp;

```

```

}

```

```

int partition (long long arr[], int low, int high)

```

```

    long long pivot = arr[high];

```

```

    int i = low - 1;

```



```

for (int j = low; j < high; j++) {
    if (arr[j] <= pivot) {
        i++;
        swap (&arr[i], &arr[j]);
    }
}
swap (&arr[i+1], &arr[high]);
return i+1;
}

```

```

void quickSort (long long arr[], int low, int high) {
    if (low < high) {
        int pi = partition (arr, low, high);
        quickSort (arr, low, pi-1);
        quickSort (arr, pi+1, high);
    }
}

```

```

int main () {
    int n;
    scanf ("%d", &n);
}

```

```

long long * sales = (long long *) malloc (n * size of (long long))
for (int i = 0; i < n; i++) scanf ("%lld", &sales[i])

```



```
quickSort ( sales, 0, n-1 );
```

```
for (int i = n-1; i >= 0; i--) {  
    printf ("%lld", sales [i]);  
    if (i != 0) printf (" ");  
}  
printf ("\n");
```

```
long long top 1 = sales [n-1];  
long long top 2 = sales [n-2];  
long long top 3 = sales [n-3];
```

```
printf ("Top 3 ; %lld %lld %lld\n", top 1, top 2, top 3);
```

```
double average = (top 1 + top 2 + top 3) / 3.0;
```

```
if (average == (long long) average {  
    printf ("Average : %lld\n", (long long) average);  
} else {
```

```
    printf ("Average : %.2f\n", average);  
}
```

```
free (sales);
```

```
return 0;
```